

Name: Sourav Choudhary

Reflection Report

Working on this project gave me a much better understanding of how real-time embedded systems operate using interrupts instead of continuous polling. At the beginning, I only had a basic understanding of Arduino programming, but by completing this task I learned how different hardware components and software logic work together to build a responsive traffic light system.

I designed the project using the **Sense–Think–Act** approach. The **Sense** part detects inputs from the vehicle, pedestrian and emergency push buttons. These inputs are handled using **Pin Change Interrupts (PCI)**, which means the Arduino can react immediately whenever a button is pressed. The **Think** part is controlled by a Finite State Machine (FSM), which decides which traffic light state should be active at any given time. Finally, the **Act** part updates the traffic lights, pedestrian indicator and buzzer according to the current state. Breaking the program into these three stages made the code much easier to understand and manage.

One of the most interesting parts of the project was learning how interrupts work. Instead of checking the buttons continuously, the program waits for an interrupt to occur and then responds quickly. I also used a **Timer1 interrupt** to keep track of the traffic light timing. This allowed the system to run without using the `millis()` function while keeping the timing accurate. To keep the program reliable, the interrupt service routines only update flags, while the main program performs the longer processing. This helped separate event handling from the main traffic control logic.

I also faced several challenges during development. One of the biggest problems was that the Arduino in Tinkercad repeatedly stopped working and displayed a red error. At first I thought the issue was caused by the program, so I spent time checking the code and debugging it. After testing the circuit one component at a time, I discovered that the actual problem was incorrect wiring on the breadboard. Once the wiring was corrected, the system worked as expected. I also had to replace the enum states with const byte values because of compatibility issues in Tinkercad.

Overall, this project improved both my programming and troubleshooting skills. I learned that writing correct code is only one part of building an embedded system, and that careful hardware testing is just as important. Completing this project has given me more confidence in using interrupts, designing finite state machines, and developing reliable real-time Arduino applications.